

LAB 2: Sensores y Actuadores

A. Sensores Parte 1

La presente evaluación será recibida sólo si se ha cumplido con los checkpoints de todas las preguntas en el **LABORATORIO** con el docente del curso.

La entrega debe constar de:

1. **Introducción** (incluir párrafo introductorio):
 1. **Marco Teórico** (definiciones **con referencias**)
 2. **Estado del Arte** (trabajos de investigación, artículos, tesis, páginas **con referencias**).
2. **Metodología** (explicación de componentes e implementos utilizados y diagramas de flujo/bloques).
3. **Desarrollo de la experiencia** (incluye discusión)
4. **Conclusiones y recomendaciones** (una por cada experiencia realizada **como mínimo**)
5. **Referencias** (formato IEEE)

Consideraciones:

- Subir únicamente el PDF del informe a Canvas, usar un link para el repositorio, incluir fotos de paso a paso de cada implementación y un vídeo mostrando el funcionamiento de las experiencias. Sin embargo, para simulaciones, considere la placa Arduino UNO disponible en Tinkercad.
- Considere una placa de Arduino MEGA2560 tal cual existe en el laboratorio.
- Solo un miembro del grupo sube el informe.
- No olvidar colocar los nombres de todos los integrantes.

Índice

SISTEMA 1: Lectura Analógica y Barra LED	4
Materiales requeridos	4
Requerimientos del sistema	4
Entregables y Evaluación	5
SISTEMA 2: Sensor de Luz Ambiental y Monitoreo de Sistema	5
Materiales requeridos	5
Requerimientos del sistema	5
Entregables y Evaluación	5
SISTEMA 3: Control de Aforo y Seguridad Ambiental	6
Materiales requeridos	6
Requerimientos del sistema	6
Entregables y Evaluación	6
ANEXO A. Códigos de Colores de las Resistencias.	7
ANEXO B. Uso del Protoboard	7
Conexionado básico	8
ANEXO C. Alimentación de protoboard con Arduino	9
ANEXO D. Potenciómetro	10
Configuración como resistencia variable (reóstato)	10
Configuración como divisor de tensión	10
ANEXO E. El Diodo LED (Light Emitting Diode)	11
ANEXO F. Instrumentos y Formas de Medición	12
Medición de Intensidad de Corriente ("I")	12
Medición de Tensión o Voltaje ("V" o "U")	12
ANEXO G. Programación con Arduino: Interfaz de Arduino	13
ANEXO H. Botón con Arduino.	14
Configuración de botón con dos puertos.	15
Activar Pull-up interno en Arduino	15
ANEXO I. Monitor Serial.	15
ANEXO J. Teclado Matricial	19
ANEXO K. Matriz de Puntos 8x8 con Controlador MAX7219	21
ANEXO L. Fococelda o Resistor Dependiente de Luz (LDR)	22
Características eléctricas típicas	23
Principio de funcionamiento	23
Uso de un LDR con Arduino	23
ANEXO M. Sensor de Proximidad	26
ANEXO N. LCD 2x16	29
ANEXO O. Lectura Análoga con Arduino	31
ANEXO P. Sensor de Movimiento PIR	33
ANEXO Q. Sensor de Humedad	37
ANEXO R. Sensor de Temperatura	38

ANEXO S. Diagramas de flujo	41
Simbología básica	41
Ejemplo de Diagrama de Flujo para un código simple.	42
ANEXO T. Diagrama de Bloques	44
Objetivo de su Uso en el Diseño Electrónico	44
Componentes Típicos	44
Ejemplo	45

SISTEMA 1: Lectura Analógica y Barra LED

Este primer subsistema tiene como objetivo introducir los conceptos de conversión analógico-digital (ADC) y el escalamiento de señales. Se implementará un indicador visual proporcional a la variación de voltaje ingresada mediante un potenciómetro, simulando el comportamiento de un control de volumen o vúmetro.



Materiales requeridos

- 01 Potenciómetro (de 10 k Ω o 100 k Ω).
- 10 Diodos LED.
- 10 Resistencias fijas de 220 Ω o 330 Ω .
- Arduino MEGA2560.

Requerimientos del sistema

Adquisición y Mapeo

El sistema debe leer continuamente el valor analógico generado por el divisor de tensión del potenciómetro (obteniendo un rango numérico de 0 a 1023) y escalar matemáticamente a un valor porcentual del 0% al 100%.

Actuación Visual

Dependiendo del valor porcentual calculado, se deberá encender una cantidad proporcional de LEDs de forma secuencial. A modo de ejemplo, si el potenciómetro se encuentra al 10% de su capacidad, equivaldrá a 1 LED encendido; al 50%, se encenderán 5 LEDs; y al 100%, los 10 LEDs deberán estar encendidos simultáneamente.

Envío de datos

Se deberá imprimir en el monitor serial el valor analógico crudo (0-1023) y su respectivo porcentaje calculado (0-100%) en tiempo real para verificar la correcta conversión lógica.

Entregables y Evaluación

1. **Checkpoint 1:** Realizar la simulación.
2. **Checkpoint 2:** Realizar la implementación física y grabar un video demostrativo.

Nota para el informe: Se debe presentar un diagrama de bloques y un diagrama de flujo del sistema implementado.

SISTEMA 2: Sensor de Luz Ambiental y Monitoreo de Sistema

Se desarrolla un sistema capaz de medir el nivel de iluminación para el control automático de luminarias, incorporando de manera simultánea la monitorización del suministro eléctrico diseñado para cámaras de vigilancia.

Materiales requeridos

- 04 Resistencias LDR (componente analógico).
- 04 Resistencias fijas de 10 kΩ.
- Diodos LED (Rojo y Azul).
- 01 Buzzer.
- 01 Botón o switch (para simular el estado de alimentación).
- Arduino MEGA2560 o placa equivalente.

Requerimientos del sistema

Iluminación Promediada y Autodiagnóstico

Se calculará el promedio de las cuatro lecturas analógicas provenientes de los LDR. Si el promedio decae por debajo de un umbral establecido, se encenderán las luminarias (representadas por un LED ROJO). Adicionalmente, si una de las lecturas permanece estática en 0 o 1023, se deberá ignorar dicho sensor para el cálculo del promedio y emitir un doble pitido de advertencia mediante el buzzer para indicar fallo en el sensor.

Monitoreo del Suministro Eléctrico

El funcionamiento óptimo del sistema de cámaras estará representado por un LED AZUL encendido de forma fija. Se emplea un switch para simular la alimentación; al detectar su desconexión (caída a GND), el sistema deberá hacer parpadear el LED AZUL y activar un tono de alarma constante en el buzzer.

Entregables y Evaluación

- **Checkpoint 3:** Realizar la simulación.
- **Checkpoint 4:** Realizar la implementación física y grabar un video demostrativo.

Nota para el informe: Se debe presentar un diagrama de bloques y un diagrama de flujo del sistema implementado.

SISTEMA 3: Control de Aforo y Seguridad Ambiental

Este ejercicio abarca el desarrollo de un dispositivo que contabiliza el ingreso y salida de personas en un recinto, integrando paralelamente el monitoreo de la calidad del aire para activar protocolos de evacuación automática en caso de contingencias (humo o fugas de gas).

Materiales requeridos

- 01 Sensor PIR (Movimiento).
- 01 Sensor de Gas MQ2.
- 01 Pantalla LCD 2x16.
- 01 Botón (simulador de salidas).
- Diodos LED (Rojo y Verde).
- 01 Buzzer.
- Arduino MEGA2560 o placa equivalente.

Requerimientos del sistema

Control de Ocupación

El sistema utilizará el sensor PIR para detectar los ingresos. Por cada detección, se incrementará un contador interno aplicando un tiempo de bloqueo (debounce) de 5 segundos para evitar lecturas duplicadas. El botón simulará las salidas, decrementando el contador. La cantidad de ocupantes se mostrará en el LCD bajo el formato "AFORO: XX". Al alcanzar el límite máximo (por ejemplo, 50 personas), la pantalla mostrará "AFORO COMPLETO" y se encenderá un LED Rojo fijo, bloqueando lógicamente cualquier nuevo ingreso.

Alarma Ambiental de Prioridad Máxima

El microcontrolador deberá leer de manera permanente la salida analógica del sensor MQ2. Si el nivel de gas o humo supera el umbral de seguridad, el sistema deberá detener el control de aforo y sobrescribir la pantalla LCD con el mensaje "PELIGRO: EVACUAR". En este estado, el buzzer emitirá un sonido de emergencia y el LED Rojo parpadeará continuamente hasta que los niveles de gas retornen a la normalidad.

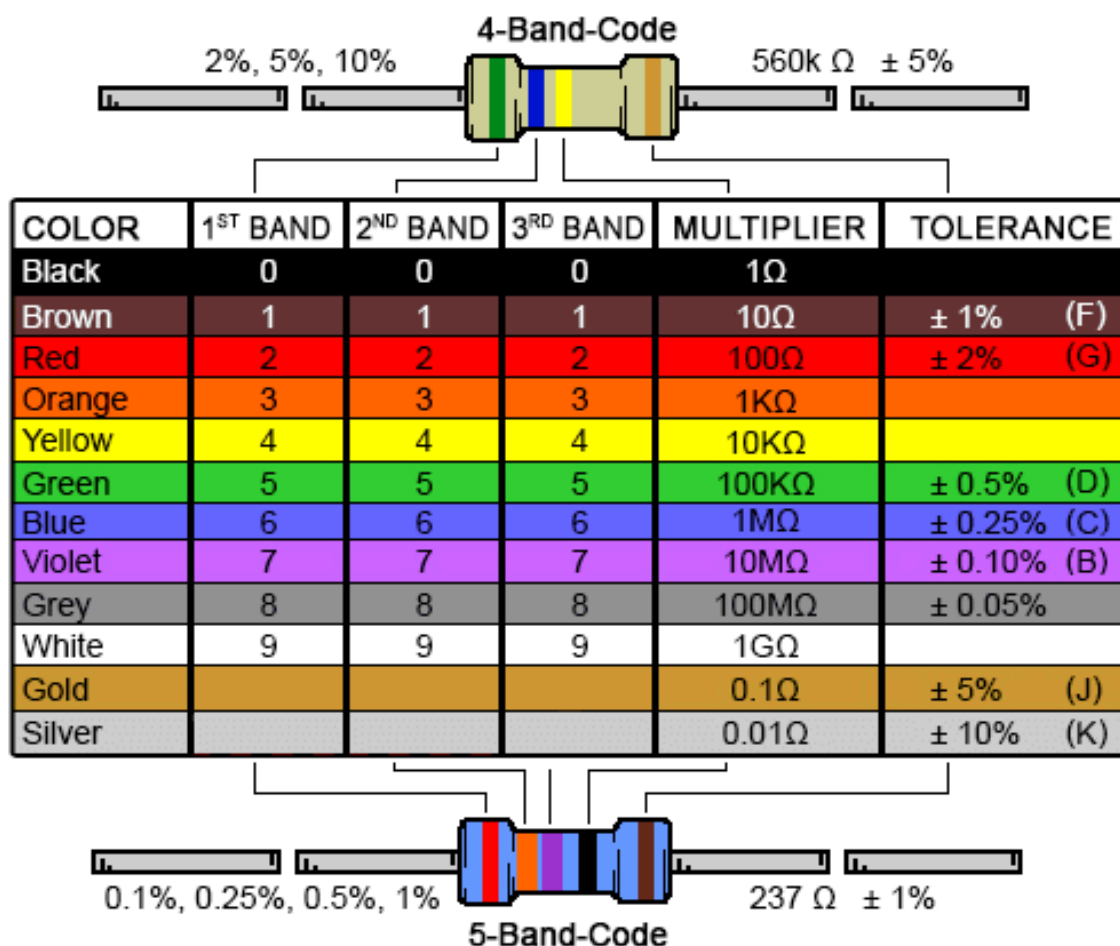
Entregables y Evaluación

- **Checkpoint 5:** Realizar la simulación.
- **Checkpoint 6:** Realizar la implementación física y grabar un video demostrativo.

Nota para el informe: Se debe presentar un diagrama de bloques y un diagrama de flujo del sistema implementado.

ANEXOS

ANEXO A. Códigos de Colores de las Resistencias.

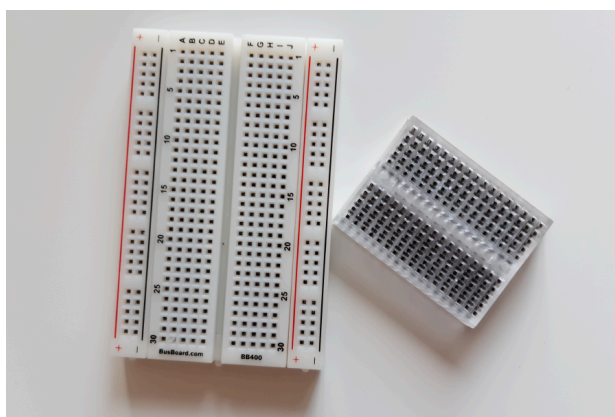


Recordar que para conectar **diodos LED** utilizar resistencias entre **100 Ohm y 330 Ohm**.

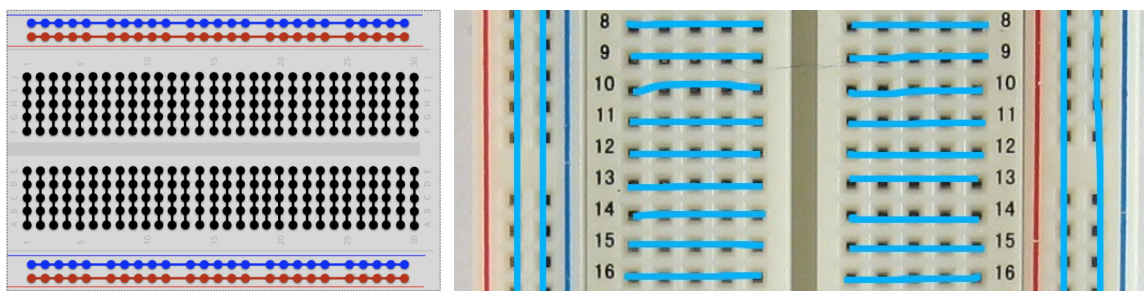
Así como para utilizar la configuración de botones pull-up y pull-down, usar resistencias de 4.7k Ohm en adelante.

ANEXO B. Uso del Protoboard

Un **protoboard**, también conocido como **breadboard**, es una herramienta utilizada para armar circuitos electrónicos de forma temporal sin necesidad de soldar los componentes. Está compuesto por una base plástica con una matriz de agujeros conectados internamente por tiras metálicas. Los componentes, como resistencias, transistores, LEDs, cables, entre otros, se insertan en los agujeros para crear conexiones eléctricas.

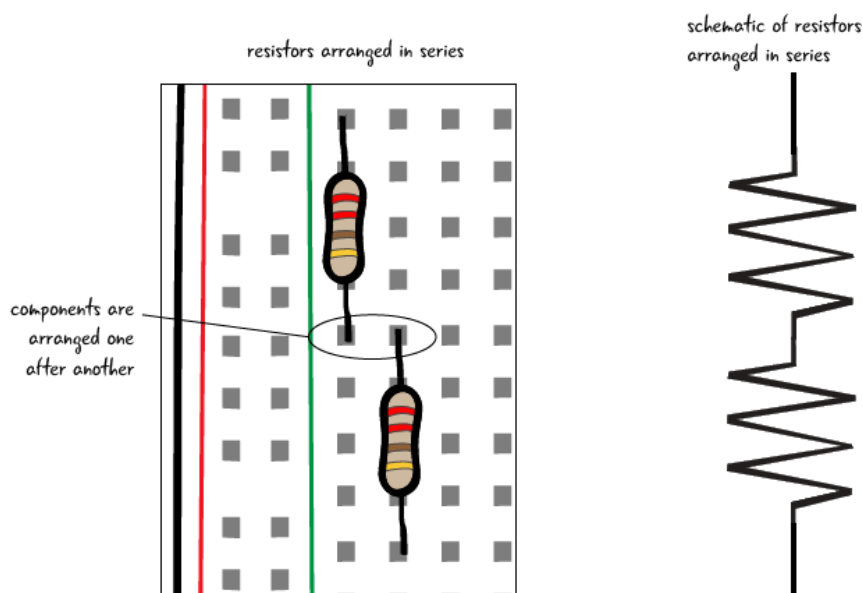


La conexiones del protoboard se dan de la manera siguiente: (1) Los orificios laterales están conectados en serie, mientras que los centrales se conectan en serie por cada columna, pero de columna a columna se encuentran en paralelo.

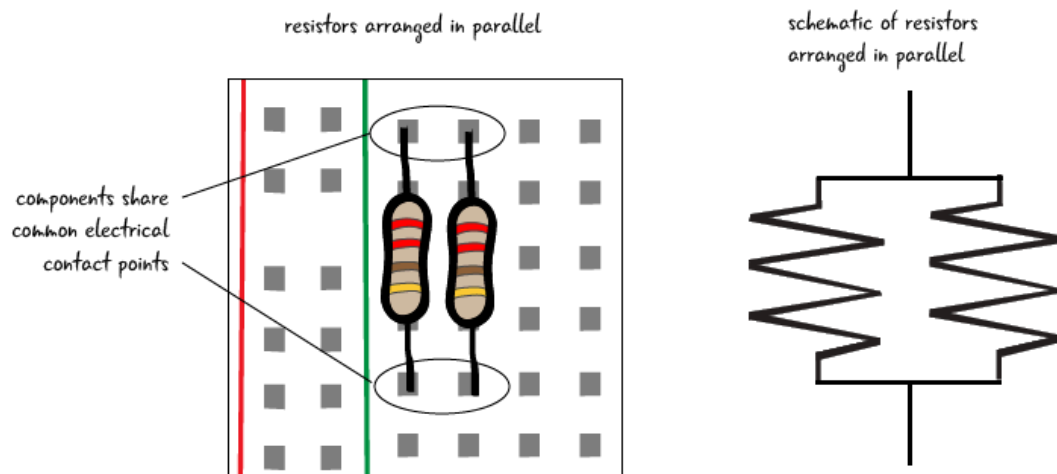


Conexionado básico

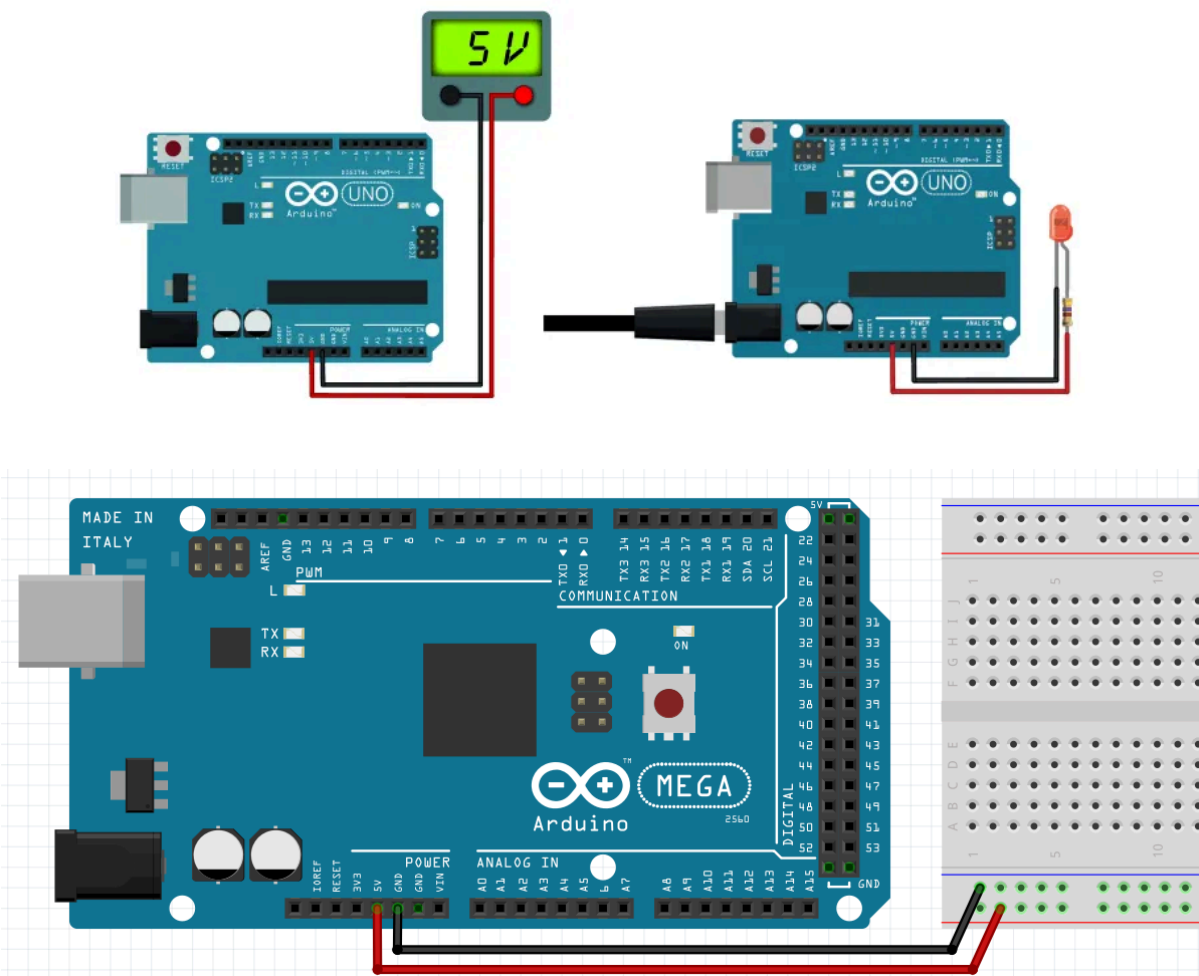
Conexión en serie



Conexión en paralelo

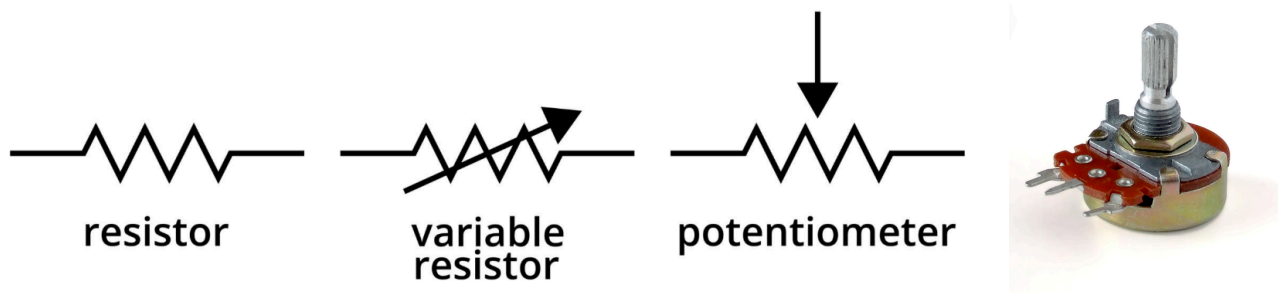


ANEXO C. Alimentación de protoboard con Arduino



ANEXO D. Potenciómetro

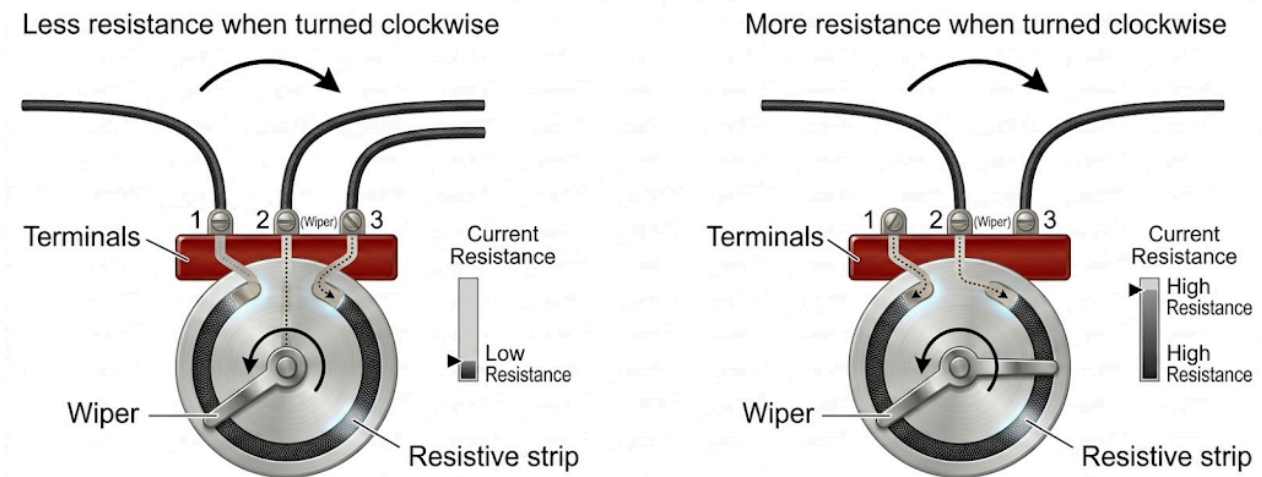
Un potenciómetro es esencialmente un resistor variable de tres terminales. Permite ajustar manualmente el valor de la resistencia, lo que lo hace ideal para controlar variables como el voltaje o la corriente en una parte específica de un circuito (por ejemplo, para ajustar referencias en simulaciones o variar la frecuencia de un reloj digital). Está compuesto por una pista resistiva (generalmente de carbón) conectada a los dos terminales extremos. Un tercer terminal (el central, o *wiper*) está conectado a un contacto móvil que se desliza sobre la pista cuando se gira la perilla.



Se usa en dos tipos de conexionado dentro de los circuitos electrónicos.

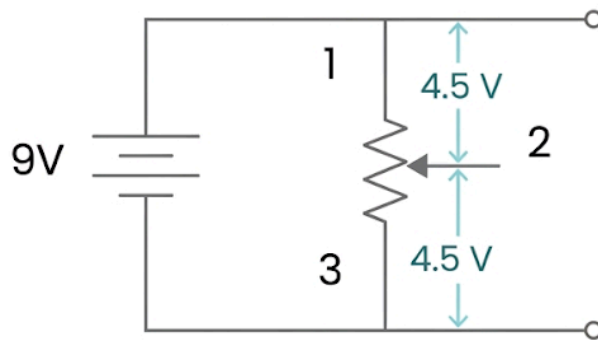
Configuración como resistencia variable (reóstato)

Si solo se utiliza un terminal extremo y el terminal central, el dispositivo se comporta como una resistencia que varía de cero a su valor nominal máximo.



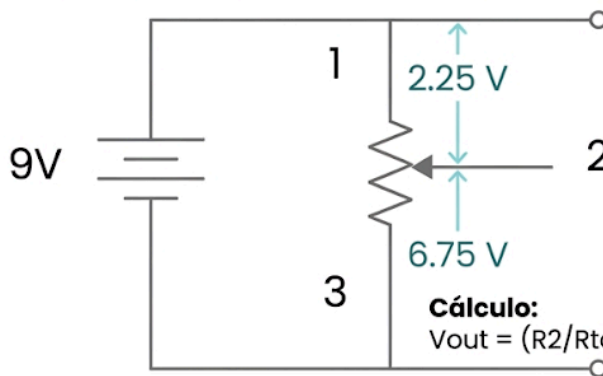
Configuración como divisor de tensión

Si se aplica un voltaje entre los dos terminales extremos, el terminal central entregará un voltaje proporcional a su posición.



**Potenciómetro
en el punto
medio.**

2: CASO DE 3/4 DE POSICIÓN



Perilla a 3/4 de su posición.

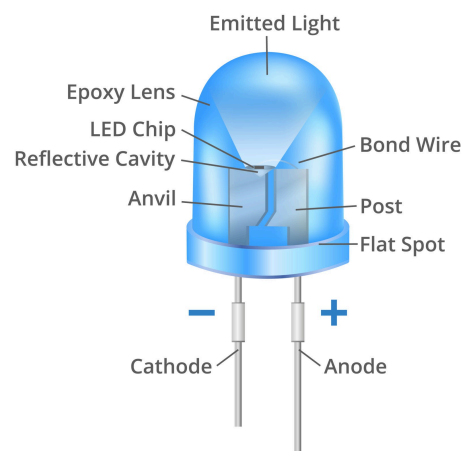
$$9V/4=2.25V$$

Cálculo:
 $V_{out} = (R_2/R_{total}) * V_{in}$

ANEXO E. El Diodo LED (Light Emitting Diode)

Un LED es un semiconductor que emite luz cuando la corriente eléctrica lo atraviesa en un sentido específico. A diferencia de las bombillas tradicionales (o de un resistor), el LED es un componente polarizado, lo que significa que la dirección en la que se conecta es crítica para que funcione (y para evitar que se queme).

Light Emitting Diode



- **Ánodo y Cátodo:** El terminal positivo se llama ánodo (usualmente la pata más larga) y el negativo se llama cátodo (la pata más corta, que suele coincidir con un lado plano en el encapsulado plástico del LED).

- Voltaje directo (Forward Voltage): Todo LED requiere un voltaje mínimo para encenderse (típicamente entre 1.8V y 3.3V, dependiendo del color).
- Regla de oro: Un LED siempre debe ir acompañado de un resistor limitador de corriente en serie. Si se conecta directamente a una fuente de alimentación mayor a su voltaje directo sin protección, absorberá demasiada corriente y se destruirá casi instantáneamente.

ANEXO F. Instrumentos y Formas de Medición

Para garantizar el correcto funcionamiento de los circuitos y la seguridad durante las sesiones prácticas, es indispensable dominar las técnicas de medición. A continuación, se detallan los procedimientos estándar para medir corriente y tensión.

Medición de Intensidad de Corriente ("I")

La intensidad de corriente indica el flujo de electrones a través de un conductor y se mide utilizando un amperímetro (ya sea en su versión analógica o como función dentro de un multímetro digital).

- Unidad de medida: Amperio [A].
- Método de conexión: El instrumento debe conectarse estrictamente en SERIE con la carga.
- Procedimiento: Para lograr esto, se debe "abrir" el circuito sobre la línea o fase que se desea medir e intercalar las puntas del amperímetro. Esto obliga a que toda la corriente pase a través del instrumento de medición.



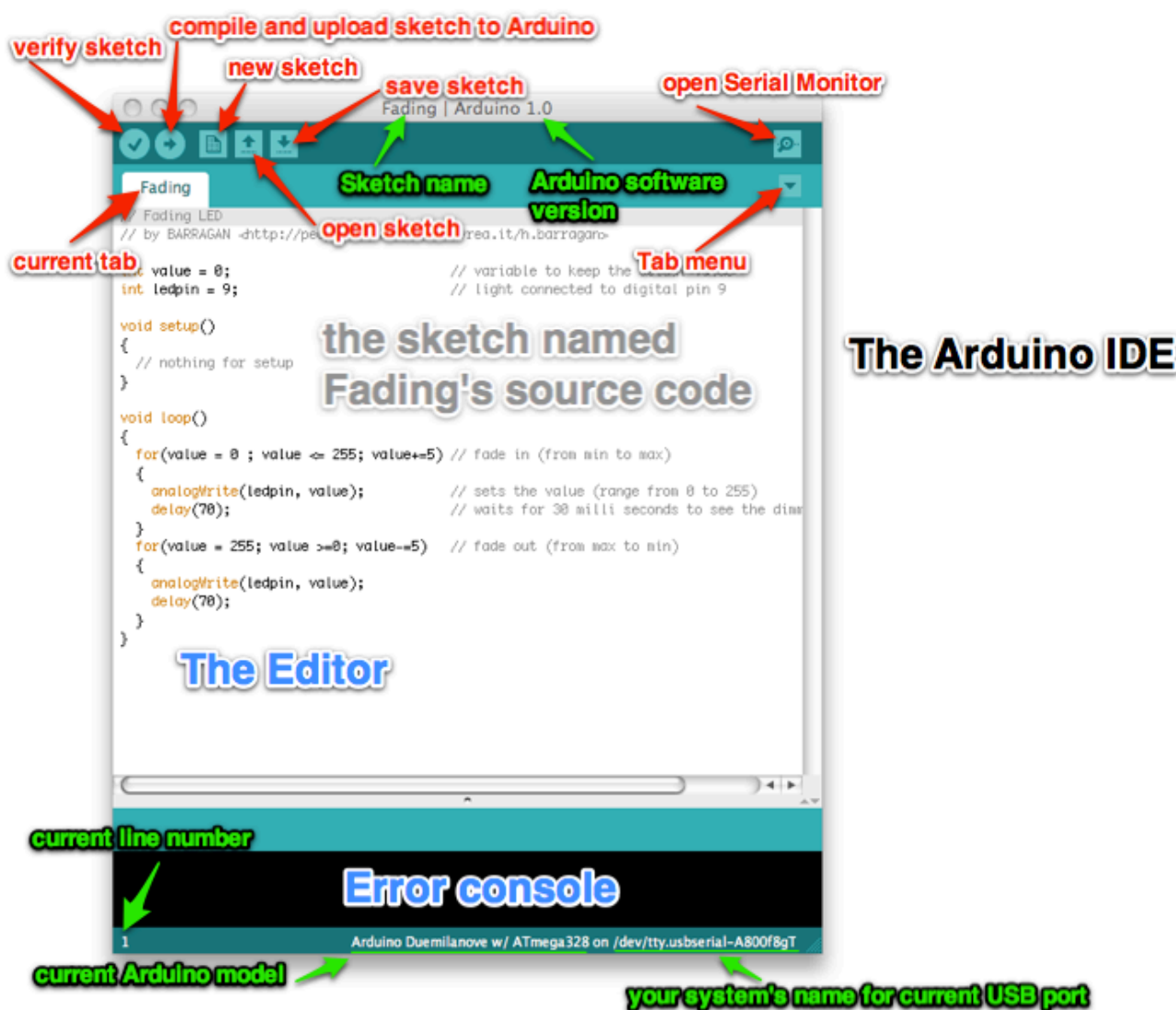
Medición de Tensión o Voltaje ("V" o "U")

La tensión representa la diferencia de potencial eléctrico entre dos puntos y se mide utilizando un voltímetro.

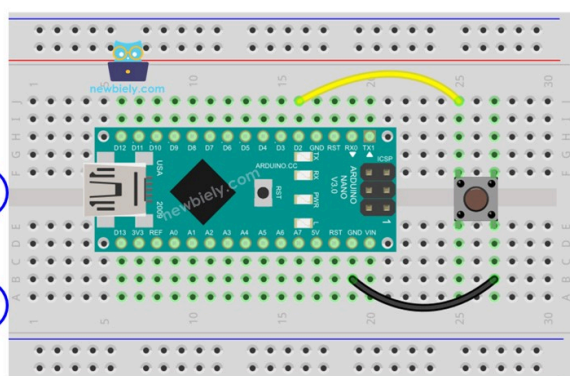
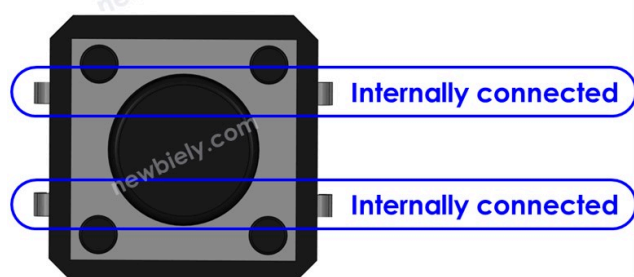
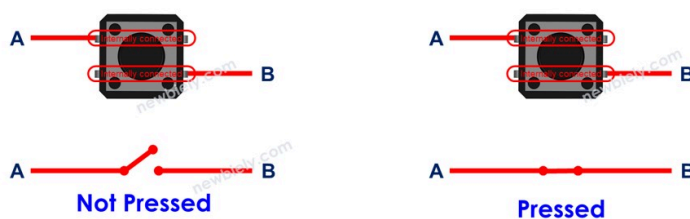
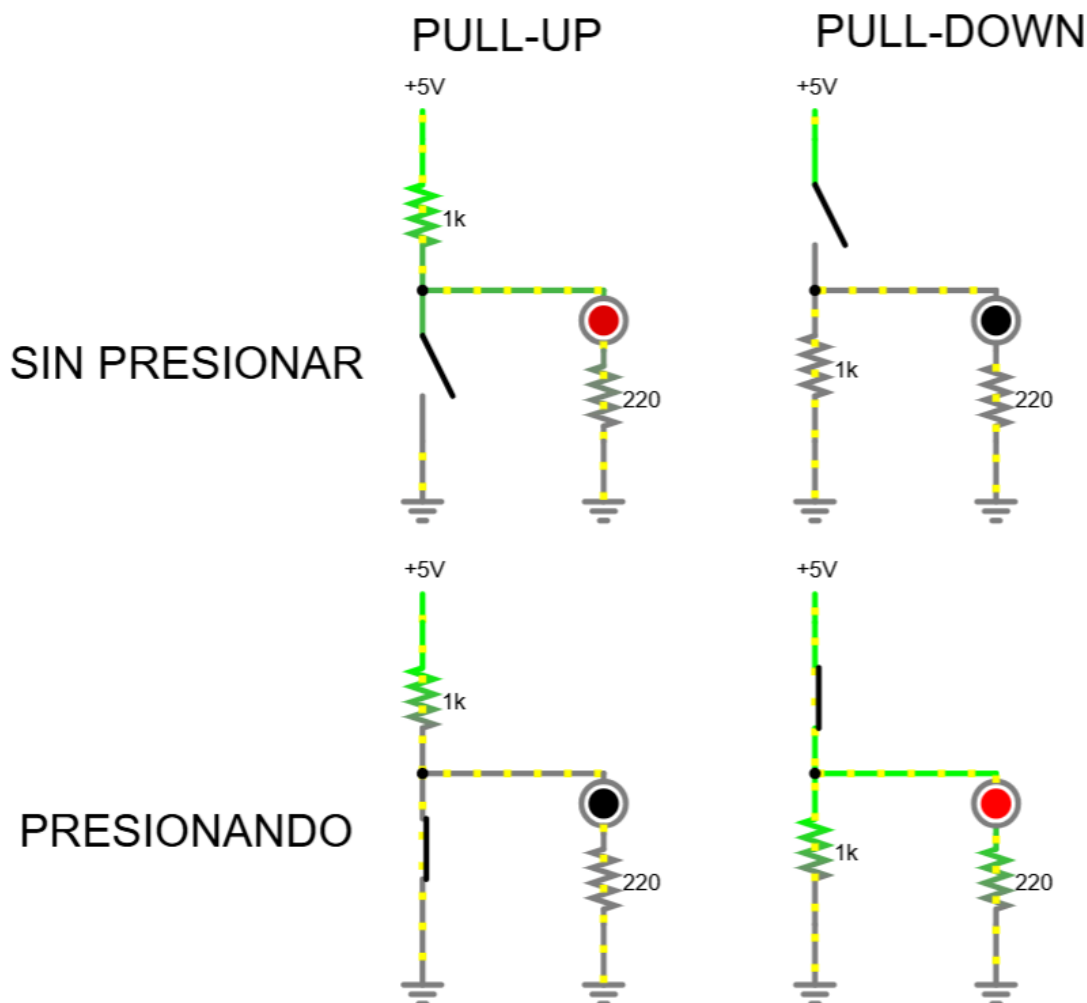
- Unidad de medida: Voltio [V].
- Método de conexión: El instrumento debe conectarse de forma PARALELA a la carga o a la fuente de alimentación (tomacorriente, barra, batería, etc.). No es necesario abrir el circuito.
- Procedimiento en redes eléctricas:
 - Circuito monofásico (220V): Se coloca una punta del instrumento sobre la línea de fase y la punta restante sobre la línea de neutro.
 - Circuito trifásico (380V): Las puntas del instrumento se deben colocar sobre cada una de las fases (para medir tensión entre fases), o entre cualquiera de las tres fases y el neutro.

Unidad de medición: VOLTIO [V]	 Símbolo General del Voltímetro
---------------------------------------	--

ANEXO G. Programación con Arduino: Interfaz de Arduino

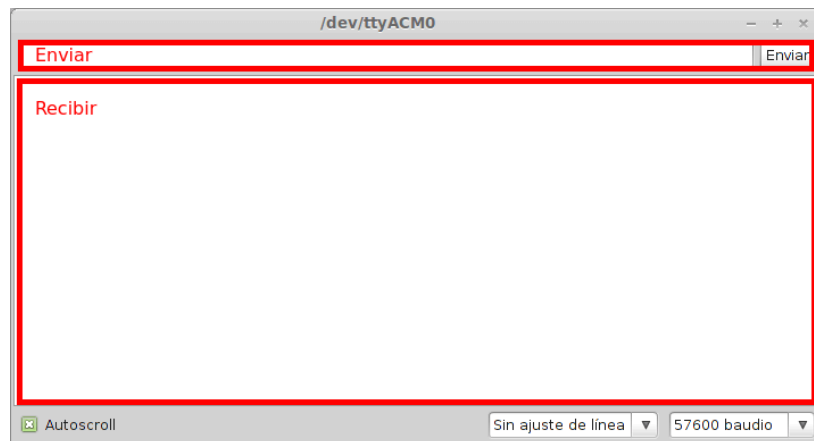


ANEXO H. Botón con Arduino.



El Monitor Serial es una herramienta integrada en el entorno de desarrollo (IDE) de Arduino que permite la comunicación bidireccional entre la computadora y el microcontrolador mediante el protocolo UART (a través del cable USB).

Esta interfaz es fundamental para la depuración de código (debugging), ya que permite al programador visualizar el estado de las variables, los resultados de los sensores en tiempo real y los mensajes de error. Así mismo, permite controlar el sistema enviando caracteres o cadenas de texto desde la computadora hacia la placa.

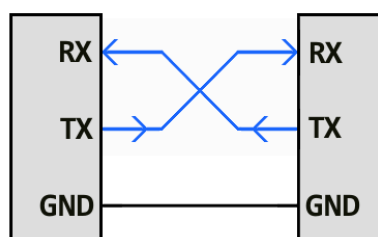


2. Principio de Funcionamiento (UART) y Conexión

La comunicación serial en Arduino se basa en hardware UART (Universal Asynchronous Receiver-Transmitter). Al ser asíncrona, no existe una señal de reloj (clock) compartida que sincronice a ambos dispositivos. En su lugar, los dispositivos se conectan cruzando sus líneas de transmisión y recepción:

- TX (Transmit): Pin por donde el dispositivo envía los datos.
- RX (Receive): Pin por donde el dispositivo recibe los datos.
- GND (Ground): Ambos dispositivos deben compartir la misma referencia de tierra para que los niveles de voltaje (0V y 5V) sean interpretados correctamente.

Nota: Cuando conectas el Arduino por USB a la computadora, un chip conversor en la placa realiza internamente esta conexión hacia los pines digitales 0 (RX) y 1 (TX).



3. El Baudrate (Velocidad de Transmisión)

Como no hay una señal de reloj que indique cuándo empieza y termina un bit, ambos dispositivos deben acordar previamente a qué velocidad van a "hablar" y "escuchar". Esta velocidad se mide en Baudios (Baud rate), que en este contexto equivale a bits por segundo (bps).

- Si configuras el Arduino a 9600 baudios, significa que enviará y leerá 9600 ceros o unos por segundo.
- La computadora (el Monitor Serial) debe estar configurada a la misma velocidad. Si el Arduino envía datos a 9600 baudios pero el Monitor Serial está configurado para "escuchar" a 115200 baudios, la computadora leerá las señales a destiempo, traduciendo los pulsos eléctricos como "basura" (caracteres extraños como ??~).

4. Funciones Principales

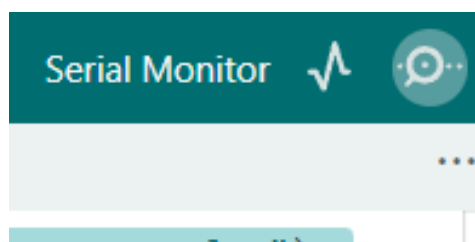
Para utilizar el puerto serial en Arduino, se emplean las siguientes funciones básicas dentro del código:

- `Serial.begin(velocidad)`: Inicializa la comunicación serial por hardware. Debe colocarse dentro del `setup()`. Los parámetros más comunes son 9600 o 115200 baudios.
- `Serial.print(dato)` y `Serial.println(dato)`: Envía datos al monitor serial. La instrucción `println` añade un salto de línea al final del mensaje.
- `Serial.available()`: Retorna el número de bytes (caracteres) disponibles para ser leídos en el buffer de recepción. Se usa típicamente en un condicional `if (Serial.available() > 0)` para asegurar que haya datos antes de leer.
- `Serial.read()`: Lee el primer byte de los datos entrantes disponibles en el buffer serial.

5. Apertura y Configuración

Para abrir el Monitor Serial en el IDE de Arduino:

1. Haga clic en el icono de la lupa ubicado en la esquina superior derecha del IDE, o presione `Ctrl + Shift + M`.
2. Asegúrese de que el menú desplegable de baudios (esquina inferior derecha de la ventana del monitor) coincida exactamente con el valor declarado en su `Serial.begin()`.



6. Ejemplo Práctico: Control de LED vía Consola

El siguiente código demuestra cómo encender y apagar el LED integrado en la placa Arduino enviando caracteres específicos mediante el Monitor Serial.

```
const int ledPin = 13; // Pin del LED integrado en la placa
char comando;          // Variable para almacenar el carácter recibido

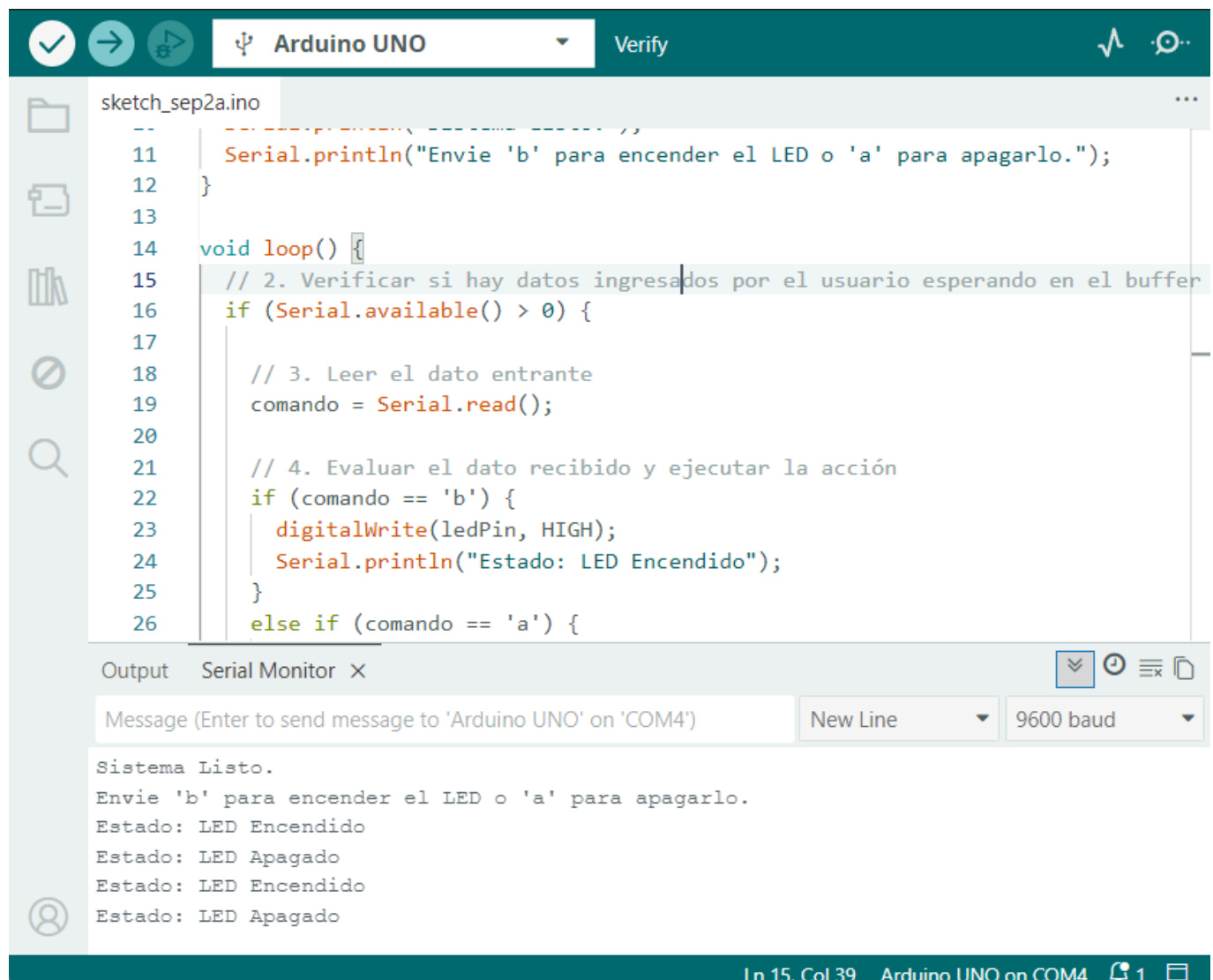
void setup() {
    // 1. Inicializar la comunicación serial a 9600 baudios
    Serial.begin(9600);
    pinMode(ledPin, OUTPUT);

    // Imprimir un mensaje inicial
    Serial.println("Sistema Listo.");
    Serial.println("Envíe 'b' para encender el LED o 'a' para apagarlo.");
}

void loop() {
    // 2. Verificar si hay datos ingresados por el usuario esperando en el buffer
    if (Serial.available() > 0) {

        // 3. Leer el dato entrante
        comando = Serial.read();

        // 4. Evaluar el dato recibido y ejecutar la acción
        if (comando == 'b') {
            digitalWrite(ledPin, HIGH);
            Serial.println("Estado: LED Encendido");
        }
        else if (comando == 'a') {
            digitalWrite(ledPin, LOW);
            Serial.println("Estado: LED Apagado");
        }
    }
}
```



```
11 Serial.println("Envie 'b' para encender el LED o 'a' para apagarlo.");
12 }
13
14 void loop() {
15   // 2. Verificar si hay datos ingresados por el usuario esperando en el buffer
16   if (Serial.available() > 0) {
17
18     // 3. Leer el dato entrante
19     comando = Serial.read();
20
21     // 4. Evaluar el dato recibido y ejecutar la acción
22     if (comando == 'b') {
23       digitalWrite(ledPin, HIGH);
24       Serial.println("Estado: LED Encendido");
25     }
26     else if (comando == 'a') {
```

Output Serial Monitor X

Message (Enter to send message to 'Arduino UNO' on 'COM4') New Line 9600 baud

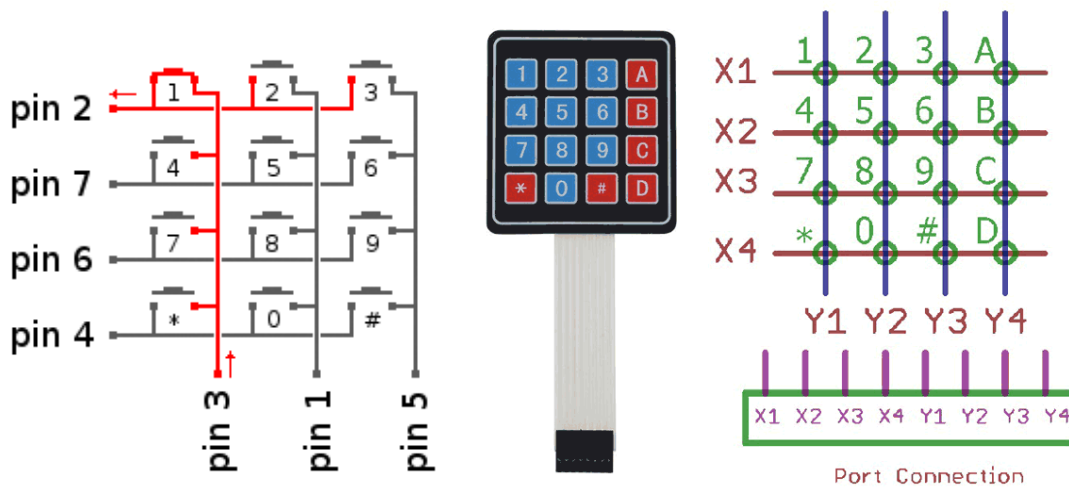
Sistema Listo.
Envie 'b' para encender el LED o 'a' para apagarlo.
Estado: LED Encendido
Estado: LED Apagado
Estado: LED Encendido
Estado: LED Apagado

Ln 15. Col 39 Arduino UNO on COM4 1

ANEXO J. Teclado Matricial

Usar referencia:

<https://controlautomaticoeducacion.com/sistemas-embebidos/arduino/teclado-matricial-keypad/>



Una forma rápida de usar un Teclado con Arduino, es valernos de su librería Keypad (<https://www.arduino.cc/reference/en/libraries/keypad/>), bastante sencilla de entender, para el caso de un teclado matricial 4x4 (KEYPAD 4x4) tenemos:

```
#include <Keypad.h>
const byte filas = 4;
const byte columnas = 4;
byte pinesFilas[] = {9,8,7,6};
byte pinesColumnas[] = {5,4,3,2};
char teclas[4][4] = {{'1','2','3','A'},
                    {'4','5','6','B'},
                    {'7','8','9','C'},
                    {'*','0','#','D'}};
```

```
Keypad teclado1 = Keypad( makeKeymap(teclas), pinesFilas, pinesColumnas, filas, columnas);
```

```
void setup() {
  Serial.begin(9600);
  Serial.println("Teclado 4x4 con Biblioteca Keypad");
  Serial.println();
}
```

```
void loop() {
  //Verifica si alguna tecla fue presionada
  char tecla_presionada = teclado1.getKey();
```

```
  //Monitor Serial
  if (tecla_presionada)
  {
    Serial.print("Tecla: ");
    Serial.println(tecla_presionada);
  }
}
```

ANEXO K. Matriz de Puntos 8x8 con Controlador MAX7219

1. Definición y Principio de Funcionamiento

Una matriz de LED 8x8 contiene 64 diodos individuales organizados en filas y columnas. Para evitar usar 64 pines del microcontrolador, se utiliza una técnica de barrido visual (multiplexado). El módulo comercial mostrado en el archivo image_d09c08.jpg simplifica enormemente este proceso al incorporar el circuito integrado MAX7219.

Este chip asume la carga de trabajo del barrido a alta frecuencia (aprovechando la persistencia retiniana del ojo humano) y se comunica con el Arduino mediante un protocolo serie síncrono muy rápido llamado SPI (Serial Peripheral Interface), requiriendo únicamente 3 pines de datos.

2. Conexiones (Pines del Módulo)

El módulo cuenta con 5 pines de entrada (IN) en un extremo y, a menudo, 5 pines de salida (OUT) en el otro, lo que permite conectar múltiples matrices en cascada.

- VCC: Alimentación (5V).
- GND: Tierra.
- DIN (Data In): Recibe los datos enviados por el Arduino.
- CS / LOAD (Chip Select): Habilita el módulo para recibir y procesar los datos.
- CLK (Clock): Señal de reloj generada por el Arduino para sincronizar la transmisión de cada bit.

3. Representación de Imágenes (Mapeo de Bits)

Cada imagen en la matriz se concibe como un conjunto de 8 filas, donde cada fila tiene 8 LEDs. En programación, esto equivale a 1 byte por fila (donde 1 es LED encendido y 0 es apagado). Una imagen completa requiere un Array de 8 bytes.

Ejemplo de diseño de un "Cuadrado":

```
1  byte cuadrado[8] = {
2      B11111111, // Fila 0
3      B10000001, // Fila 1
4      B10000001, // Fila 2
5      B10000001, // Fila 3
6      B10000001, // Fila 4
7      B10000001, // Fila 5
8      B10000001, // Fila 6
9      B11111111  // Fila 7
10 };
11 // Nota: También se pueden expresar en formato
12 // hexadecimal (ej. 0xFF en vez de B11111111).
```

4. Librería Recomendada y Código Base Para abstraer la complejidad del protocolo SPI, se recomienda instalar y usar la librería <LedControl.h> desde el gestor de librerías del IDE de Arduino.

Funciones principales de LedControl:

- `LedControl lc = LedControl(DIN, CLK, CS, Num_Matrices);` : Inicializa el objeto.
- `lc.shutdown(0, false);` : Despierta la matriz (por defecto inicia en modo ahorro).
- `lc.setIntensity(0, brillo);` : Ajusta el brillo (de 0 a 15).
- `lc.clearDisplay(0);` : Apaga todos los LEDs.
- `lc.setRow(0, fila, valorByte);` : Dibuja un byte entero en una fila específica.

Ejemplo de inicialización:

```
1  #include <LedControl.h>
2
3  // Pines: DIN=11, CLK=13, CS=10. El '1' indica que solo hay 1 matriz conectada.
4  LedControl lc = LedControl(11, 13, 10, 1);
5
6  void setup() {
7      lc.shutdown(0, false);      // Enciende la matriz 0
8      lc.setIntensity(0, 8);      // Establece brillo medio (0 a 15)
9      lc.clearDisplay(0);         // Limpia la pantalla
10 }
```

ANEXO L. Fotocelda o Resistor Dependiente de Luz (LDR)

Un LDR (Light Dependent Resistor), también conocido como fotocelda o fotoresistor, es un dispositivo electrónico pasivo cuya resistencia eléctrica varía inversamente con la intensidad de la luz incidente sobre su superficie. Es decir, a medida que aumenta la luminosidad, la resistencia disminuye, y cuando la iluminación se reduce, la resistencia aumenta.

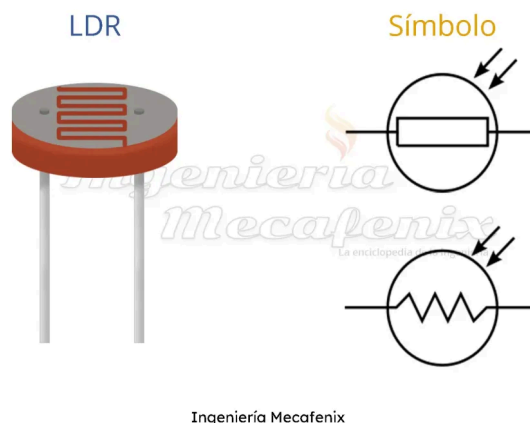
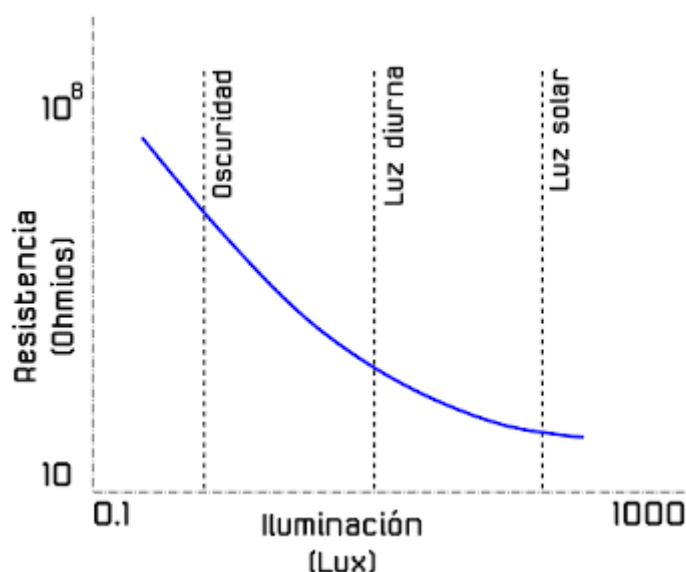


Imagen y diagrama esquemático de un LDR



Curva de iluminación de un LDR: Resistencia vs. iluminación.

Características eléctricas típicas

- Rango de resistencia en oscuridad: 1 MΩ o superior.
- Rango de resistencia bajo iluminación intensa: 100 Ω a 1 kΩ.
- Tensión máxima de operación: 150 V a 250 V (dependiendo del modelo).
- Corriente máxima: alrededor de 1 mA a 5 mA.

Principio de funcionamiento

El LDR está fabricado con materiales semiconductores, típicamente sulfuro de cadmio (CdS) o selenio, que presentan el efecto fotoeléctrico. Al incidir luz sobre el material, los fotones liberan electrones, aumentando así la conductividad del semiconductor y disminuyendo su resistencia.

Uso de un LDR con Arduino

Para medir la intensidad de la luz mediante un **Arduino**, el **LDR** se debe conectar formando un **divisor de voltaje** junto con una resistencia fija. El Arduino leerá la variación de voltaje a través de una de sus entradas analógicas.

Esquema de conexión

- Un terminal del LDR se conecta a **5V** del Arduino.
- El otro terminal del LDR se conecta a un punto común con:
 - Un extremo de la **resistencia de 10 kΩ**, cuyo otro extremo va a **GND**.
 - Una conexión hacia un pin **analógico** de Arduino (por ejemplo, **A0**).

Esto forma un **divisor de voltaje**, donde el voltaje leído en A0 depende de la iluminación.

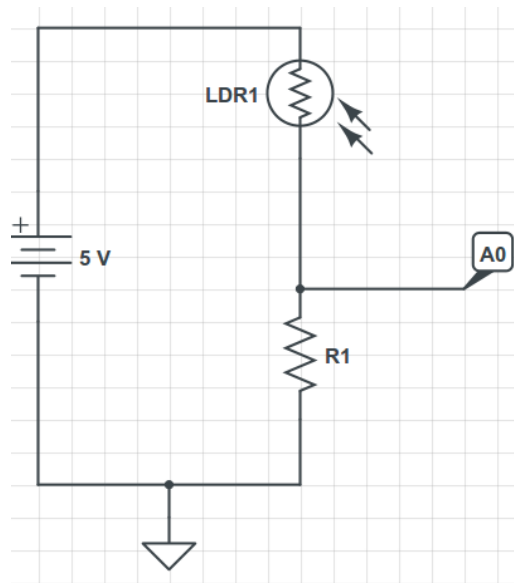


Diagrama de conexión de un LDR en configuración de divisor de voltaje.

Código básico de lectura:

```
int sensorPin = A0; // Pin conectado al divisor de voltaje

int sensorValue = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  sensorValue = analogRead(sensorPin);

  Serial.println(sensorValue);

  delay(500); // Medio segundo entre lecturas
}
```

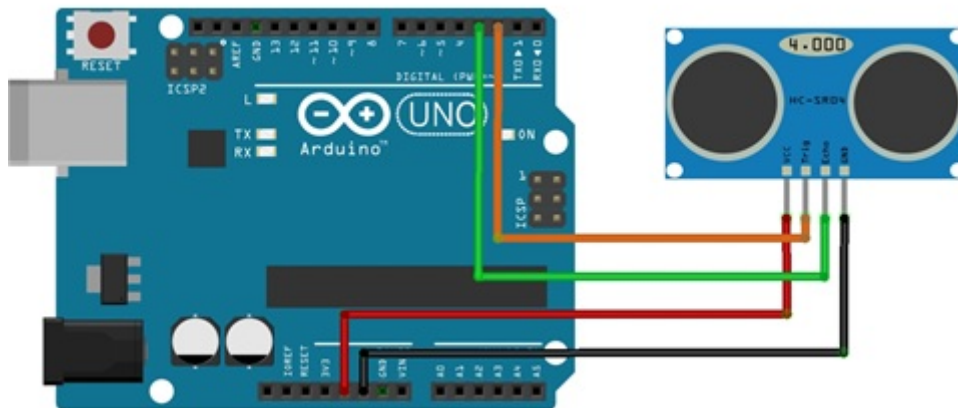
Funcionamiento

- **analogRead(A0)** devuelve un valor entre **0** (0V) y **1023** (5V).
- Mayor iluminación $\Rightarrow$ menor resistencia LDR $\Rightarrow$ mayor voltaje a la entrada analógica A0.
- Menor iluminación $\Rightarrow$ mayor resistencia LDR $\Rightarrow$ menor voltaje a la entrada analógica A0.

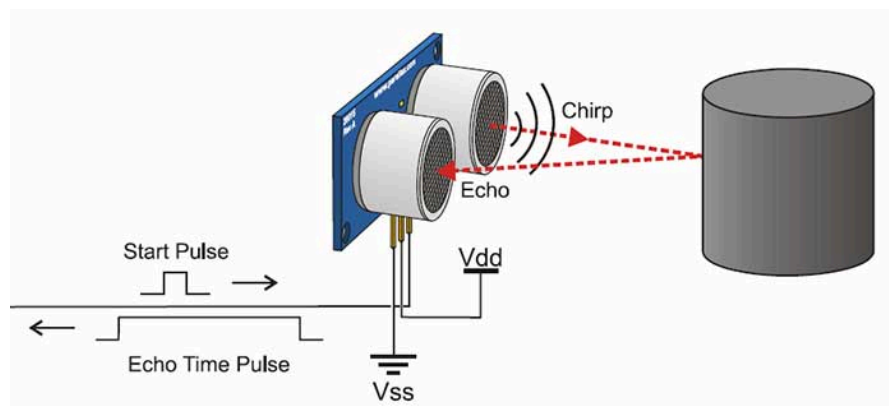
ANEXO M. Sensor de Proximidad

Para el correcto uso del sensor de proximidad se recomienda la guía disponible en línea de Naylamp.

[Enlace a tutorial de Sensor de Proximidad Ultrasónico](#)



Tenga en cuenta que la información es referencial. Nuestro trabajo utiliza como microcontrolador principal el Arduino MEGA.



Partimos de la siguiente formula:

$$Velocidad = \frac{distancia\ recorrida}{tiempo}$$

Donde **Velocidad** es la velocidad del sonido 340m/s, pero usaremos las unidades en cm/us pues trabajaremos en centímetros y microsegundos, **tiempo** es el tiempo que demora en llegar el ultrasonido al objeto y regresar al sensor, y la **distancia recorrida** es dos veces la distancia hacia el objeto, reemplazando en la formula tenemos:

$$\frac{340m}{s} \times \frac{1s}{1000000us} \times \frac{100cm}{1m} = \frac{2d}{t}$$
$$d(cm) = \frac{t(us)}{59}$$

Ejemplo de código:

```
const int Trigger = 2; //Pin digital 2 para el Trigger del sensor

const int Echo = 3; //Pin digital 3 para el Echo del sensor


void setup() {

  Serial.begin(9600); //inicializamos la comunicación

  pinMode(Trigger, OUTPUT); //pin como salida

  pinMode(Echo, INPUT); //pin como entrada

  digitalWrite(Trigger, LOW); //Inicializamos el pin con 0
}


void loop()

{

  long t; //timepo que demora en llegar el eco

  long d; //distancia en centimetros


  digitalWrite(Trigger, HIGH);
```

```
delayMicroseconds(10);    //Enviamos un pulso de 10us  
  
digitalWrite(Trip, LOW);  
  
t = pulseIn(Echo, HIGH); //obtenemos el ancho del pulso  
d = t/59;                //escalamos el tiempo a una distancia en cm  
  
Serial.print("Distancia: ");  
  
Serial.print(d);    //Enviamos serialmente el valor de la distancia  
  
Serial.print("cm");  
  
Serial.println();  
  
delay(100);        //Hacemos una pausa de 100ms  
}
```

ANEXO N. LCD 2x16

Para el correcto uso del LCD 2x16 se recomienda la guía disponible en línea de Naylamp.

Nota. Usar la librería LiquidCrystal.

[Guía de uso del LCD1602](#)

Funciones de la librería.

- **LiquidCrystal(rs, en, d4, d5, d6, d7)**

Función constructor, crea una variable de la clase LiquidCrystal, con los pines indicados.

- **begin(cols, rows)**

Inicializa el LCD, es necesario especificar el número de columnas (cols) y filas (rows) del LCD.

- **clear()**

Borra la pantalla LCD y posiciona el cursor en la esquina superior izquierda (posición (0,0)).

- **setCursor(col, row)**

Posiciona el cursor del LCD en la posición indicada por col y row (x,y); es decir, establecer la ubicación en la que se mostrará posteriormente texto escrito para la pantalla LCD.

- **write()**

Escribir un carácter en la pantalla LCD, en la ubicación actual del cursor.

- **print()**

Escribe un texto o mensaje en el LCD, su uso es similar a un Serial.print

- **scrollDisplayLeft()**

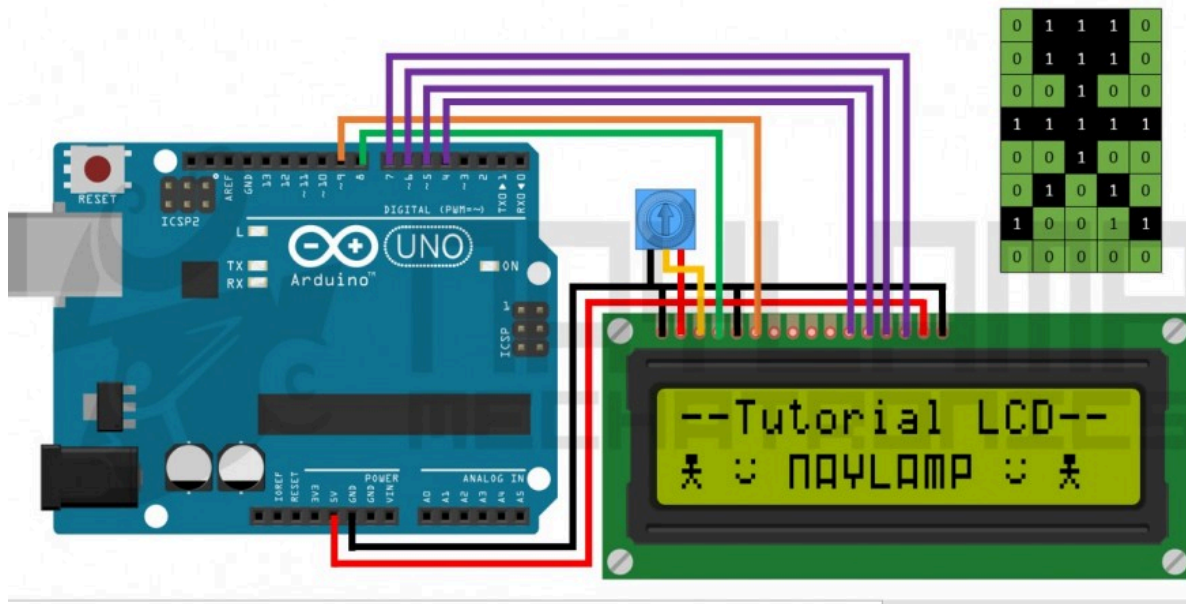
Se desplaza el contenido de la pantalla (texto y el cursor) un espacio hacia la izquierda.

- **scrollDisplayRight()**

Se desplaza el contenido de la pantalla (texto y el cursor) un espacio a la derecha.

- **createChar (num, datos)**

Crea un carácter personalizado para su uso en la pantalla LCD. Se admiten hasta ocho caracteres de 5x8 píxeles (numeradas del 0 al 7). Donde: num es el número de carácter y datos es una matriz que contienen los píxeles del carácter. Se verá un ejemplo de esto mas adelante.



Recomendación.

Utilizar la menor cantidad de pines disponibles.

```
#include <LiquidCrystal.h>
```

```
//Crear el objeto LCD con los números correspondientes (rs, en, d4, d5, d6, d7) SOLO USANDO  
6 PINES.
```

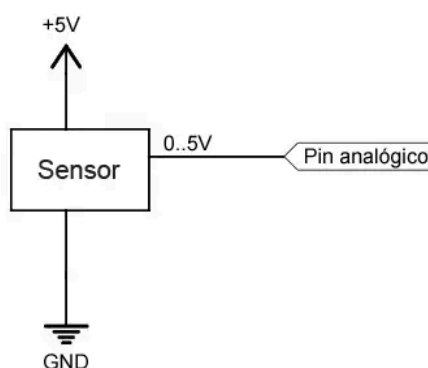
```
LiquidCrystal lcd(8, 9, 4, 5, 6, 7);
```

ANEXO O. Lectura Analógica con Arduino

En entradas anteriores hemos visto cómo emplear las entradas digitales de nuestro Arduino. En esta entrada vamos a ver las entradas analógicas, su funcionamiento y características.

Las entradas analógicas funcionan de una forma similar a las entradas digitales, por lo que en la práctica el montaje y código final son muy similares.

Una señal analógica es una magnitud que puede tomar cualquier valor dentro de un intervalo $-V_{cc}$ y $+V_{cc}$. Por ejemplo, una señal analógica de tensión entre 0V y 5V podría valer 2,72V, o 3.41V (o cualquier otro valor con cualquier número de decimales). Por otro lado, recordemos que una señal digital de tensión teórica únicamente podía registrar dos valores, que denominamos LOW y HIGH (en el ejemplo, 0V o 5V).

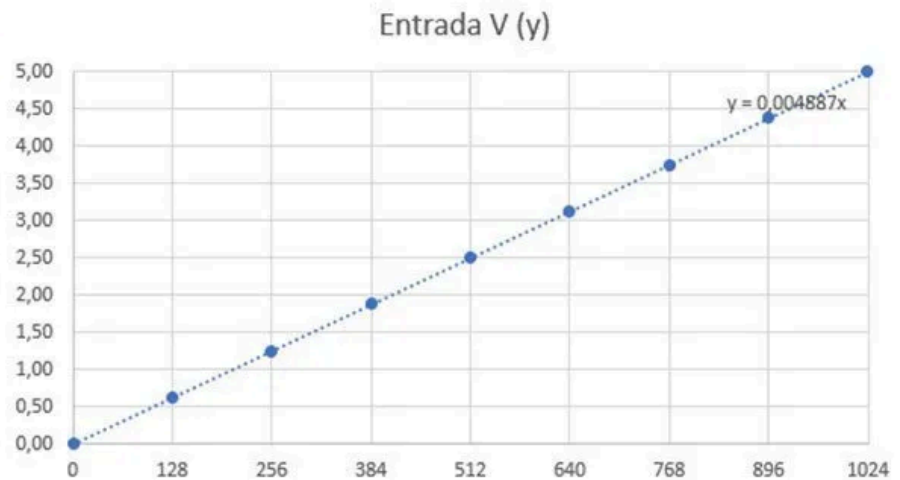


Se utiliza el comando `analogRead()`

```
sensorValue = analogRead(sensorPin);
```

Donde el valor retornado por `sensorValue` es un número entre 0 y 1024 (dependiendo de la resolución de bits del ADC del MCU usado).

y = 0,004887x		
N°	ADC (x)	Entrada V (y)
0	0	0,00
1	128	0,63
2	256	1,25
3	384	1,88
4	512	2,50
5	640	3,13
6	768	3,75
7	896	4,38
8	1023	5,00



Enlaces de referencia.

[Entradas Analógicas en Arduino](#)

[analogRead\(\) - Arduino](#)

ANEXO P. Sensor de Movimiento PIR

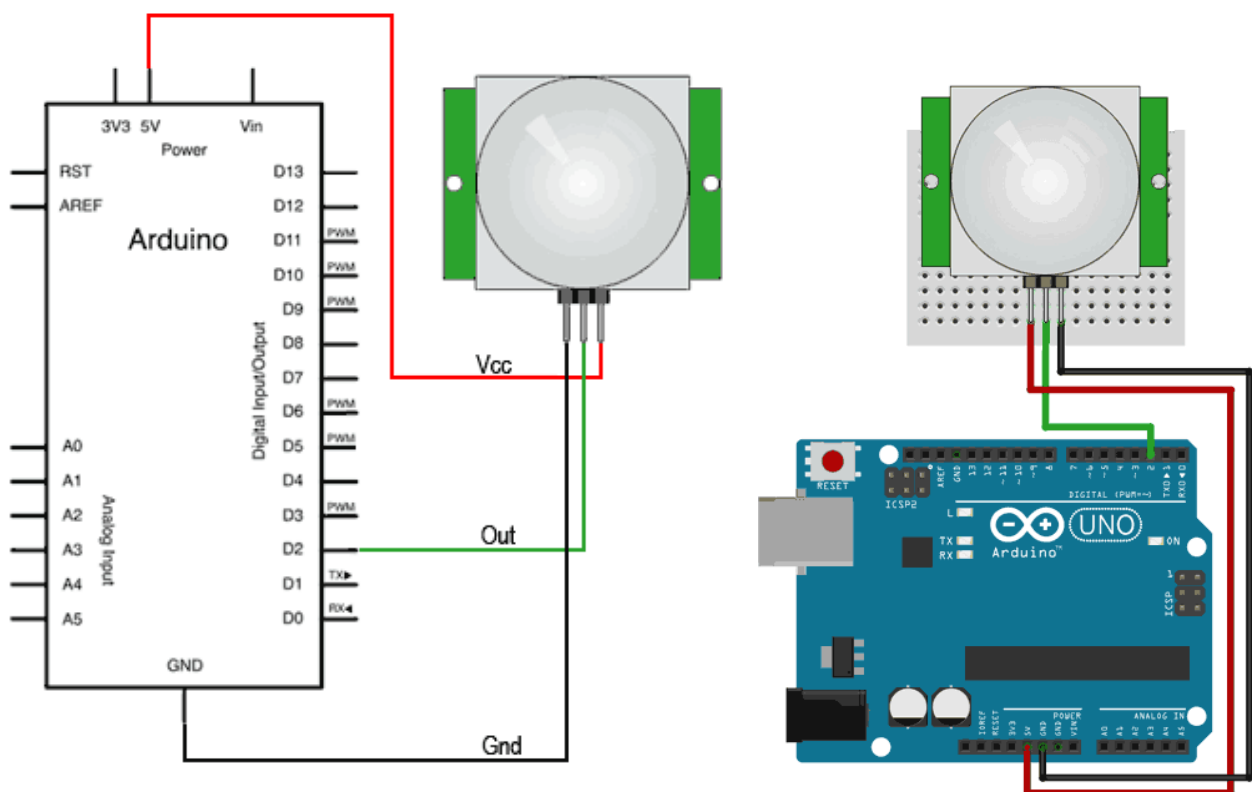
UN PIR (Infrarrojos pasivos) sensor de movimiento es un dispositivo que detecta el movimiento midiendo los cambios en radiación infrarroja. Detecta específicamente los niveles de radiación infrarroja emitida por cuerpos y objetos calientes, incluido el cuerpo humano.



Sensor de movimiento PIR.

Enlace de Referencia:

[Enlace a tutorial de Sensor PIR + Arduino](#)



Ejemplo de conexionado.

{cwilliams.garias}@utec.edu.pe

CS5055: Internet of Things

WEEK 04



Ejemplo de código.

```
const int LEDPin= 13;

const int PIRPin= 2;


void setup()
{
    pinMode(LEDPin, OUTPUT);
    pinMode(PIRPin, INPUT);
}


void loop()
{
    int value= digitalRead(PIRPin);

    if (value == HIGH)
    {
        digitalWrite(LEDPin, HIGH);
        delay(50);
        digitalWrite(LEDPin, LOW);
        delay(50);
    }
    else
    {
        digitalWrite(LEDPin, LOW);
    }
}
```

{cwilliams.garias}@utec.edu.pe

CS5055: Internet of Things

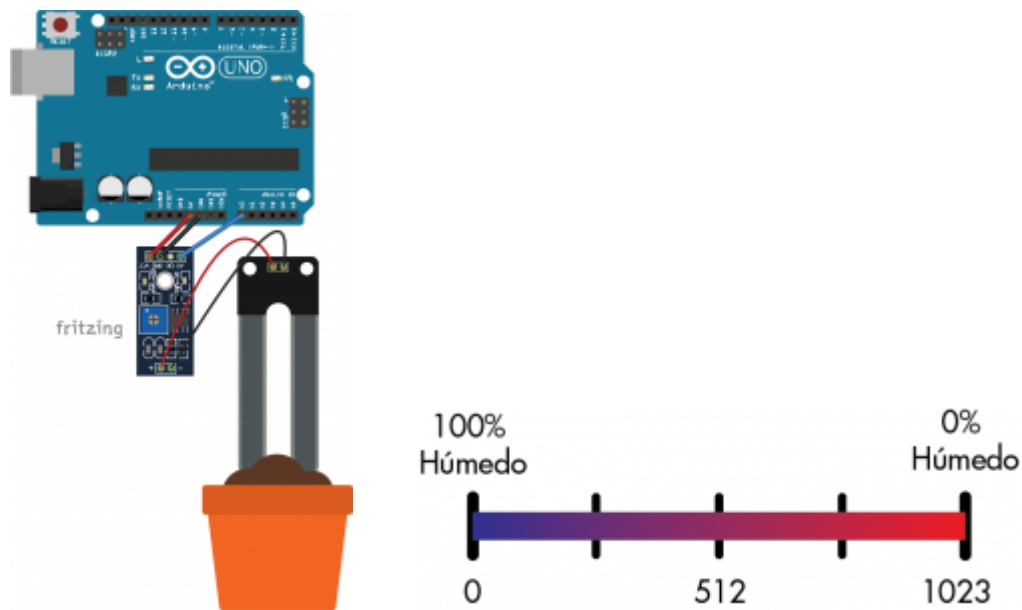
WEEK 04



ANEXO Q. Sensor de Humedad

Referencia.

Sensor de Humedad con Arduino



Escala del sensor de humedad.

Código Ejemplo

```
int SensorPin = A0;

void setup() {
  pinMode(7,OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int humedad = analogRead(SensorPin);
  Serial.println(humedad);
  if(humedad>=460)
```

```
{  
    digitalWrite(7,LOW);  
}  
  
else  
  
{  
    digitalWrite(7,HIGH);  
}  
  
delay(1000);  
}
```

ANEXO R. Sensor de Temperatura

Enlace de Referencia:

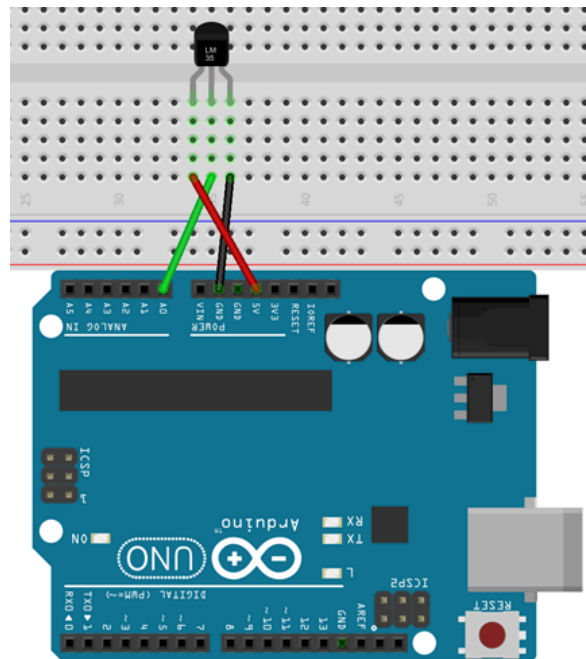
[Sensor de Temperatura con Arduino](#)

[Guia PDF de Sensor de Temperatura](#)

Nota.

Este sensor utiliza una fórmula para convertir el valor analógico medido a temperatura.

$$\text{Temperatura} = \text{Valor} * 5 * 100 / 1024$$



Código Ejemplo

```
// Declaracion de variables globales

float tempC; // Variable para almacenar el valor obtenido del sensor (0 a 1023)

int pinLM35 = 0; // Variable del pin de entrada del sensor (A0)

void setup() {

    // Configuramos el puerto serial a 9600 bps

    Serial.begin(9600);

}

void loop() {

    // Con analogRead leemos el sensor, recuerda que es un valor de 0 a 1023
```

```
tempC = analogRead(pinLM35);

// Calculamos la temperatura con la fórmula
tempC = (5.0 * tempC * 100.0)/1024.0;

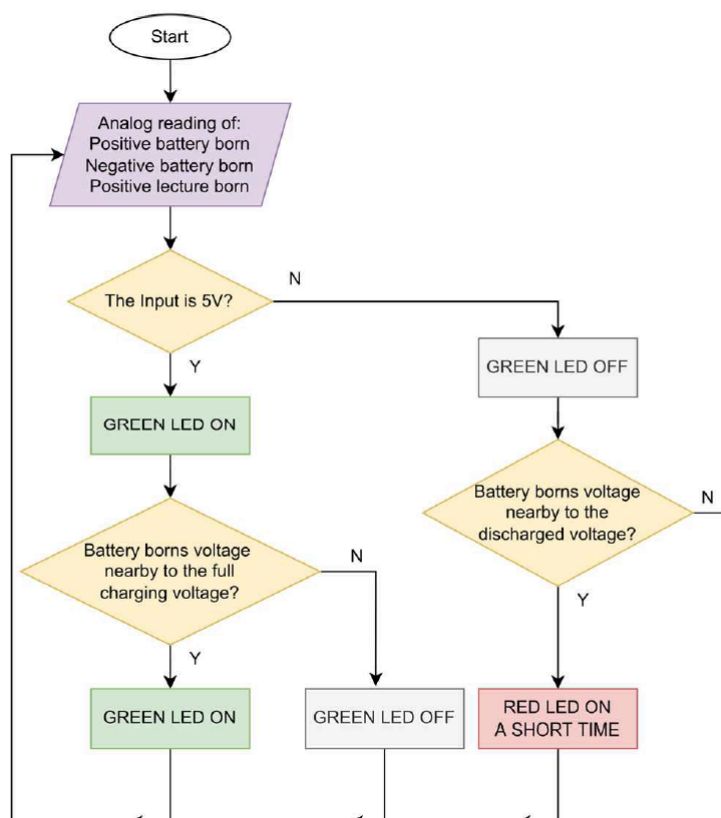
// Envía el dato al puerto serial
Serial.print(tempC);

// Salto de línea
Serial.print("\n");

// Esperamos un tiempo para repetir el loop
delay(1000);
}
```

ANEXO S. Diagramas de flujo

Los **diagramas de flujo** son representaciones gráficas que describen de manera secuencial los pasos o procesos necesarios para llevar a cabo una tarea específica. Su principal objetivo es facilitar la comprensión, el análisis y la comunicación de procesos complejos mediante el uso de **símbolos normalizados** y conectores que indican el flujo lógico de las operaciones.



Simbología básica

A continuación, se describen los símbolos más comunes utilizados en los diagramas de flujo:

- **Inicio / Fin:** Representado por un óvalo. Indica el punto de entrada o salida del proceso.
- **Proceso:** Representado por un rectángulo. Indica una acción, tarea o conjunto de operaciones.
- **Decisión:** Representado por un rombo. Indica un punto de bifurcación basado en una condición lógica (sí/no).
- **Entrada / Salida:** Representado por un paralelogramo. Indica operaciones de ingreso de datos o presentación de resultados.
- **Conector:** Círculo pequeño que permite enlazar diferentes partes del diagrama cuando este no puede ser representado de forma continua.

Símbolo	Nombre	Función
	Inicio / Final	Representa el inicio y el final de un proceso
	Línea de Flujo	Indica el orden de la ejecución de las operaciones. La flecha indica la siguiente instrucción.
	Entrada / Salida	Representa la lectura de datos en la entrada y la impresión de datos en la salida
	Proceso	Representa cualquier tipo de operación
	Decisión	Nos permite analizar una situación, con base en los valores verdadero y falso

Ejemplo de Diagrama de Flujo para un código simple.

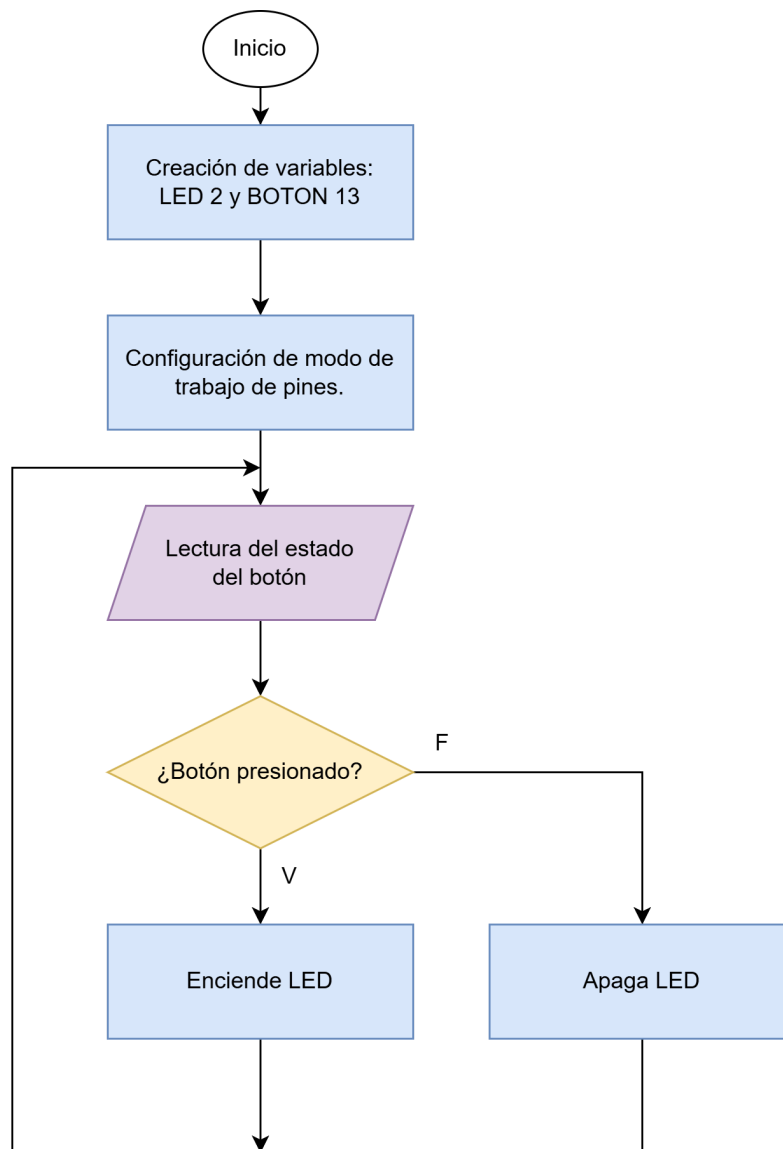
Ejemplo sencillo de código para Arduino que enciende un LED cuando se presiona un botón. Este ejemplo asume que tienes un botón conectado al pin digital 2 y un LED al pin digital 13.

```
const int botonPin = 2;    // Pin donde está conectado el botón
const int ledPin = 13;     // Pin donde está conectado el LED
int estadoBoton = 0;       // Variable para leer el estado del botón

void setup() {
  pinMode(botonPin, INPUT); // Configura el pin del botón como entrada
  pinMode(ledPin, OUTPUT);  // Configura el pin del LED como salida
}

void loop() {
  estadoBoton = digitalRead(botonPin); // Lee el estado del botón

  if (estadoBoton == HIGH) {
    digitalWrite(ledPin, HIGH); // Enciende el LED si el botón está presionado
  } else {
    digitalWrite(ledPin, LOW);  // Apaga el LED si el botón no está presionado
  }
}
```



ANEXO T. Diagrama de Bloques

Un **diagrama de bloques** es una representación gráfica de un sistema electrónico en la que se describen sus principales funciones o subsistemas mediante bloques rectangulares interconectados. Cada bloque representa una función específica, como amplificación, filtrado, conversión o control, sin detallar su implementación interna. Esta herramienta se utiliza ampliamente en la etapa de diseño conceptual para visualizar el flujo de señales, la jerarquía funcional del sistema y la interacción entre los distintos módulos.

Objetivo de su Uso en el Diseño Electrónico

El propósito de los diagramas de bloques es proporcionar una visión estructurada y abstracta del sistema a diseñar. Facilitan:

- La **planificación del diseño** a nivel macro.
- La **comunicación entre diseñadores**, al ofrecer un lenguaje común.
- La **identificación de módulos reutilizables** o estándar.
- La **definición de interfaces** entre subsistemas.

Componentes Típicos

Un diagrama de bloques está compuesto por:

- **Bloques funcionales:** Representan unidades lógicas del sistema (p. ej., fuente de alimentación, amplificador, ADC, microcontrolador).
- **Líneas de conexión:** Indican el flujo de señales o energía entre los bloques.
- **Etiquetas o señales:** Denotan entradas, salidas o parámetros críticos (p. ej., Vcc, Vin, GND, CLK, UART).

Ejemplo

